

Lecture 3: Policies, Timing, Populations

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 13, 2026

University of Toronto

Policy Function

Policy Function: c^*

The policy function is the **argmax** version of the **max** Bellman operator:

$$\mathcal{T}v(b) = \max_{c \in [0, b]} u(c) + \beta v(R(b - c) + z) \quad (\text{Bellman Operator})$$

$$c^*(b) = \arg \max_{c \in [0, b]} u(c) + \beta v(R(b - c) + z) \quad (\text{Policy Function})$$

- Once the true V is solved, $\arg \max$ gives you the optimal policy c^* .
- More deeply, any v gives you some policy function.
 - v can be thought of as representing beliefs about the value of being in various states.

Marginal Propensity to Consume (MPC)

The slope $\partial c^* / \partial b$ is the **Marginal Propensity to Consume (MPC)**

Generally (theoretically and empirically) higher for poorer households, low for rich.

Can be thought of as a function of how **constrained** households are.

Conventions

Conventions

- Now that we have some intuition, let's set up some conventions that will pay off later.
- Think of the state variable z as **human capital**, following a **Markov process** Π
- Period income is zw , where w is the wage level. For now, we'll normalize $w = 1$.
- The **state** is now (b, z) .
- On computer, each V is now a **matrix**. Rows are wealth b , columns are human capital z .

Stages

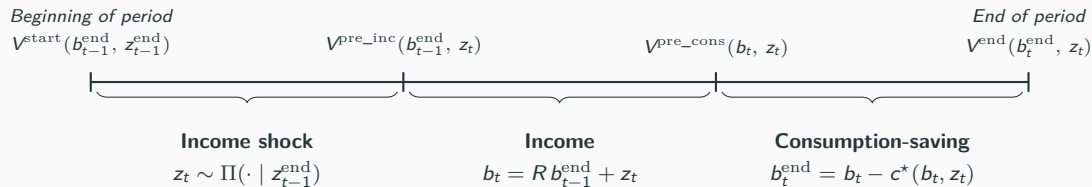
Each **model feature** can be thought of as a **Stage**:

1. Income Shock
2. Income
3. Consumption-Saving

Each **Stage** is a **simple sub-period**.

- More on this later, but **functional composability** of stages is analogous to the **additive composability** of continuous-time household models.
- This can allow us to build **complex** models by composing **Stages**.

Within-Period Timeline



Each **Stage** has different **state variables** and a different **intermediate value function**.

We can iterate V^{end} backward one stage at a time.

Stages

Stage 1 — Income Shock

- $\Pi_{ij} = \Pr(z_t = j \mid z_{t-1}^{\text{end}} = i)$, row-stochastic.
- **Backward** (value functions).

$$V^{\text{start}} = V^{\text{pre_inc}} \Pi^\top.$$

Each $(b^{\text{end}}, z_{t-1}^{\text{end}})$ entry of V^{start} is an expectation over z_t given z_{t-1}^{end} .

(Note: To see why matrix multiplication can represent backwards-iteration of V through a Markov chain, see standard references on Markov chains.)

Stage 2 — Income

This stage just represents the **receipt** of income,

$$b_t = R b_{t-1}^{\text{end}} + z_t,$$

effectively a change-of-variables.

Explicitly, the backwards step is,

$$V^{\text{pre-inc}}(b_{t-1}^{\text{end}}, z) = V(R b_{t-1}^{\text{end}} + z, z).$$

In the example code, we do this by **reinterpolating** V at the new points $R b_{t-1}^{\text{end}} + z$.

Stage 3 — Consumption-Saving

This stage represents the argmax step,

$$b_t^{\text{end}} = b_t - c^*(b_t, z_t).$$

Explicitly, the backwards step is,

$$V(b, z) = \max_{b^{\text{end}} \in b_{\text{grid}}} u(b - b^{\text{end}}) + \beta V^{\text{end}}(b^{\text{end}}, z).$$

Here, we fold in the time discounting by β in the consumption-saving stage, but in general it can be thought of as really a separate stage.

Backwards Iteration

```
function bellman_operator(V_end, params)
    (; β) = params
    # Stage 3
    V_pre_cons = consumption_saving_backward(V_end, params)

    # Stage 2
    V_pre_inc = income_backward(V_pre_cons, params)

    # Stage 1
    V_start = income_shock_backward(V_pre_inc, params)

    # Passage of time
    V_end_new = β .* V_start
    return V_end_new
end
```

Backwards iteration of V is just **functional composition** of the backwards-iteration functions of each stage.

Simulating a Population

Heterogeneity for (Almost) Free

- We have been thinking of V and c^* as the value and policy of a single household at different **possible** states.
- But we can also think of V and c^* as the value and policy of **contemporaneous** households in a **population** with different idiosyncratic states.
- Since we have already computed V on the whole grid, this lets us add heterogeneity very cheaply.
- We just need to think about how to **represent** and **simulate** a population of households.

A Population is a Vector (or Matrix)

We can represent the *population* as a discrete distribution λ over the wealth grid:

$$\lambda_i = \text{share of households at } b_i.$$

That is, just like V , it is a vector (or a matrix) over the discretized state space.

(Note: Matrices are vectors.)

Aggregate Moments

- Aggregate welfare is then an inner product: $\bar{V} = \sum_i V(b_i) \lambda_i$. Same for $\bar{C}$, $\bar{K}$, etc.
- Indeed, **any** moment of the distribution (e.g. aggregate capital, consumption, etc.) can be thought of as an inner product against λ .

Forward and Backward, All Three Stages

	Backward (value)	Forward (distribution)
Stage 1	$V^{\text{start}} = V^{\text{pre_inc}} \Pi^\top$	$\lambda^{\text{post}} = \lambda^{\text{pre}} \Pi$
Stage 2	$V^{\text{pre_inc}}(b^{\text{end}}, z) = V^{\text{pre_cons}}(R b^{\text{end}} + z, z)$	$(b^{\text{end}}, z) \rightarrow (R b^{\text{end}} + z, z)$
Stage 3	$V^{\text{pre_cons}}(b, z) = \max_{b^{\text{end}}} u(b - b^{\text{end}}) + V^{\text{end}}(b^{\text{end}}, z)$	$(b, z) \rightarrow (b - c^*(b, z), z)$
Dynamic Boundary Condition	$V^{\text{end}}_t = \beta V^{\text{start}}_{t+1}$	$\Lambda^{\text{start}}_t = \Lambda^{\text{end}}_{t+1}$
Steady State	$V^{\text{end}} = \beta V^{\text{start}}$	$\Lambda^{\text{start}} = \Lambda^{\text{end}}$

Backward: $V^{\text{end}} \rightarrow V^{\text{pre_cons}} \rightarrow V^{\text{pre_inc}} \rightarrow V^{\text{start}}$.

Forward: $\Lambda^{\text{start}} \rightarrow \Lambda^{\text{pre_inc}} \rightarrow \Lambda^{\text{pre_cons}} \rightarrow \Lambda^{\text{end}}$.

Stage 1: Markov Income Shock

Forward:

```
#  $\lambda$  is an  $N_b \times N_z$  matrix: rows index wealth, columns index z state.  
# Markov along the z dimension is a single matrix multiplication.  
result =  $\lambda$  * P_z
```

- The mass at (b_i, z_j) flows to $(b_i, z_{j'})$ with weight $\Pi_{jj'}$.
- Markov along one dimension is just a matrix multiplication.

simulate_population

```
function simulate_population(b_inds_new, params,  $\lambda_0$ ; T = 200)
    path, cur = [copy(  $\lambda_0$ )], copy(  $\lambda_0$ )
    for t in 1:T
        cur = income_shock_forward(cur, params.P_z)           # Stage 1
        cur = income_forward(cur, params)                     # Stage 2
        cur = consumption_saving_forward(cur, b_inds_new, params) # Stage 3
        push!(path, copy(cur))
    end
    return path
end
```

Forward simulation is just **functional composition** of the forward-simulation functions of each stage.

Solving for Steady State

1. Guess v and λ .
2. Iterate v backwards until it converges to V .
3. Simulate λ forward until it converges to Λ .

(Note: The conditions under which λ converges are subtler than for V .)